

Lista de Exercícios 01

Sistemas Operacionais

Prof. Pedro Botelho

27 de abril de 2026

1. Quais são as duas principais funções de um sistema operacional?
2. Instruções relacionadas ao acesso a dispositivos de E/S são tipicamente instruções privilegiadas, isto é, podem ser executadas em modo núcleo, mas não em modo usuário. Dê uma razão de por que essas instruções são privilegiadas.
3. Por que a abstração de recursos é importante para os desenvolvedores de aplicações? Ela tem alguma utilidade para os desenvolvedores do próprio sistema operacional?
4. Qual é a diferença entre modo núcleo e modo usuário? Explique como ter dois modos distintos ajuda no projeto de um sistema operacional.
5. De acordo com os conceitos de Sistemas Operacionais apresentados, existem duas visões principais que definem a sua função na arquitetura do computador: a visão Top-Down e a visão Bottom-Up. Assinale a alternativa correta em relação a essas visões.
 - a) Ambas as visões assumem que o Sistema Operacional deve executar diretamente na camada de hardware sem realizar nenhuma distinção entre o modo núcleo e o modo usuário.
 - b) A visão Top-Down era exclusiva dos sistemas operacionais de computadores mainframe da segunda geração e deixou de ser utilizada nos sistemas atuais.
 - c) O foco da visão Bottom-Up é prover uma interface gráfica (GUI) amigável para que o usuário final não precise interagir com linhas de comando complexas.
 - d) Na visão Top-Down, o SO atua como uma máquina estendida, fornecendo abstrações do hardware e interfaces simples para os programas, enquanto na visão Bottom-Up ele atua como um gerenciador de recursos físicos.
 - e) A visão Top-Down enxerga o Sistema Operacional exclusivamente como um alocador de memória RAM, não tendo nenhuma relação com abstrações de armazenamento ou dispositivos de E/S.
6. Para balancear velocidade, capacidade e custo, a arquitetura dos computadores utiliza diversas camadas físicas que formam a “Hierarquia de Memória”. Baseado na estrutura apresentada, como essas tecnologias se organizam e se comparam?

- a) Os registradores da CPU possuem o menor tempo de acesso e capacidade muito restrita, enquanto as unidades de disco entregam o maior tempo de acesso (maior lentidão) compensado por capacidades massivas.
 - b) Os registradores localizam-se na base da pirâmide e oferecem dezenas de Terabytes de espaço de memória não-volátil a um custo muito baixo.
 - c) O disco magnético é a camada de armazenamento com menor tempo de acesso (maior rapidez), sendo constantemente acessado de forma instantânea e direta pela CPU em seus cálculos lógicos.
 - d) A memória Cache possui o menor tempo de acesso (1 ns) de todo o sistema e substitui a necessidade de manter registradores dentro da CPU.
 - e) A Memória Principal (RAM) é hierarquicamente a mais lenta e a mais barata, sendo utilizada de forma puramente sequencial apenas quando o Disco Rígido atinge o limite máximo.
7. Os sistemas operacionais modernos utilizam diferentes modos de operação para garantir a segurança e a estabilidade da máquina. Sobre o “modo núcleo” (kernel) e o “modo usuário”, assinale a afirmação verdadeira.
- a) A transição de um programa do modo usuário para o modo núcleo (chaveamento de modo) ocorre exclusivamente quando o computador é reiniciado manualmente pelo usuário.
 - b) Em modo núcleo, o processador proíbe a execução de instruções mais complexas, reservando o poder máximo de processamento estritamente para os aplicativos no modo usuário.
 - c) O sistema operacional opera em modo núcleo, possuindo acesso completo e direto ao hardware, enquanto os programas comuns rodam em modo usuário, limitados a um subconjunto seguro de instruções.
 - d) Aplicativos como navegadores web e reprodutores de vídeo devem obrigatoriamente executar em modo núcleo para garantir alta performance de multimídia e acesso irrestrito à internet.
 - e) O modo usuário permite que qualquer aplicativo execute diretamente instruções físicas de leitura e gravação no disco rígido sem a necessidade de solicitar ao Sistema Operacional.
8. Para que um computador consiga interagir com o mundo externo, este utiliza Dispositivos de Entrada e Saída (E/S). A gestão destes periféricos envolve componentes físicos e lógicos. Assinale a alternativa que define corretamente a função de um “Driver” (controlador de dispositivo lógico).
- a) O Driver é uma peça universal de hardware que elimina a necessidade de barramentos físicos como PCIe, SATA e USB.

- b) Os Drivers são componentes estritamente de segurança, desenvolvidos para bloquear a leitura ou escrita não autorizada na RAM do utilizador.
 - c) O Driver atua como o componente mecânico primário, realizando fisicamente as ações, como a movimentação da cabeça de leitura nos discos rígidos.
 - d) O Driver é o chip de silício soldado na motherboard (placa-mãe) responsável por fornecer a energia elétrica ao periférico conectado.
 - e) O Driver é um módulo de código (programa) desenvolvido especificamente para “conversar” com o controlador físico do dispositivo, permitindo a interação entre ele e o Sistema Operativo.
9. O Sistema Operativo atua como um gestor de recursos, permitindo que diversos programas utilizem o hardware da máquina ao mesmo tempo através da “Multiplexagem” (ou multiplexação). Sobre os dois tipos principais de multiplexagem abordados, assinale a alternativa correta.
- a) Na multiplexagem no tempo, o recurso é dividido fisicamente, como a memória RAM, onde cada programa recebe uma fatia do hardware simultaneamente.
 - b) A multiplexagem no espaço ocorre quando os programas se revezam rapidamente no uso de um único recurso (como a CPU), criando a ilusão de execução paralela.
 - c) A multiplexagem no tempo é exclusiva para discos rígidos, enquanto a multiplexagem no espaço é utilizada estritamente para impressoras de rede.
 - d) A memória principal é um exemplo clássico de multiplexagem no espaço, pois diferentes programas recebem partes distintas do recurso para nele residirem em simultâneo.
 - e) Os Sistemas Operativos modernos abandonaram a multiplexagem no espaço, utilizando apenas a multiplexagem no tempo para gerir a execução em múltiplos núcleos de CPU.
10. De acordo com os princípios de projeto em Sistemas Operacionais, qual é a principal distinção conceitual entre “Política” e “Mecanismo”?
- a) Políticas disparam eventos externos assíncronos de hardware, e mecanismos tratam de eventos intencionais de software.
 - b) Políticas são estritamente implementadas em hardware para gerenciar interrupções, enquanto mecanismos rodam no espaço do usuário.
 - c) Políticas e mecanismos são sinônimos utilizados na API do SO para descrever rotinas de bibliotecas.
 - d) Mecanismos gerenciam o isolamento da memória virtual e as políticas realizam as chamadas de sistema para acesso ao núcleo.

- e) Políticas decidem o que deve ser feito e representam aspectos abstratos de alto nível (ex: decidir a quantidade de memória para uma aplicação), enquanto mecanismos são os procedimentos de baixo nível sobre como as ações serão executadas (ex: como iniciar um processo).
11. O hardware e o software se comunicam constantemente através de eventos que desviam o fluxo do processador. Assinale a alternativa que classifica corretamente as Interrupções, Exceções e Traps:
- a) O Trap é um evento gerado por periféricos de rede, a Exceção é solicitada por bibliotecas do usuário e a Interrupção é um erro de divisão por zero.
 - b) A Interrupção é um evento externo e assíncrono (ex: clique do mouse); a Exceção é um evento interno e síncrono (ex: divisão por zero); e o Trap é uma exceção intencional ativada por uma instrução de software (ex: instrução SYSCALL).
 - c) Interrupções e Traps são eventos puramente síncronos, gerados unicamente por erros em tempo de execução das aplicações.
 - d) A Interrupção desvia a execução através de um evento interno; a Exceção ocorre de forma assíncrona; e o Trap isola o processador para segurança.
 - e) Todos são eventos externos mascaráveis utilizados pelas portas de I/O para interromper o escalonador do núcleo.
12. Para garantir a segurança e a estabilidade, os sistemas operacionais modernos operam com níveis de privilégio distintos (como o kernelspace e o userspace). Sobre esses espaços de isolamento, é correto afirmar que:
- a) O nível núcleo possui acesso amplo a todas as instruções do processador, portas e memória, enquanto o nível usuário opera de forma restrita; tentativas inválidas do usuário de acessar recursos privilegiados geram exceções.
 - b) Tanto o núcleo quanto os aplicativos de usuário compartilham os mesmos privilégios para acelerar as operações de escrita no disco.
 - c) Aplicações rodando no nível de usuário não podem interagir de nenhuma forma com o nível de núcleo, o que garante o isolamento perfeito dos processos.
 - d) O núcleo restringe seu próprio acesso aos registradores de uso geral para proteger o hardware de falhas provocadas pelo sistema operacional.
 - e) O nível de usuário possui acesso direto a um subconjunto restrito de memória, mas acesso amplo a todas as portas de E/S físicas.
13. Aplicações rotineiramente precisam ler e escrever em arquivos, criar processos ou enviar pacotes de rede. Como essas aplicações, rodando no nível de usuário, conseguem invocar um serviço protegido que reside no nível do núcleo?

- a) A aplicação envia uma interrupção de hardware (IRQ) diretamente pela placa de rede, forçando o SO a ler o arquivo.
 - b) O código faz chamadas regulares na mesma pilha do kernel, utilizando saltos (JUMP) para o endereço de memória do núcleo.
 - c) Utilizando bibliotecas como a `unistd.h`, que alteram dinamicamente a arquitetura do processador para liberar privilégios root.
 - d) Através de um comando que realiza a troca direta de contexto na memória RAM, contornando a proteção de memória do processador.
 - e) Através do acionamento de uma função Trap, que executa a troca segura do modo de usuário para o modo núcleo, acessando o serviço desejado por meio de uma interface abstrata.
14. Considere um editor de código-fonte (como o VS Code) rodando no nível de usuário de um sistema operacional moderno. Durante o seu uso, o aplicativo executa uma série de ações distintas. Analise as operações abaixo:

- I. Realizar a contagem de quantas linhas existem no código que o usuário acabou de digitar e está armazenado na memória RAM.
- II. Solicitar a criação de um novo processo filho para rodar o compilador em segundo plano.
- III. Alterar a cor de uma palavra específica no texto alterando uma variável interna (string) do programa.
- IV. Salvar o arquivo de código atual permanentemente no disco rígido (ex: utilizando `fprintf()`).
- V. Ler as teclas que o usuário está pressionando no teclado para digitar o código.

Quais das operações acima exigem, necessariamente, que o aplicativo realize uma Chamada de Sistema (Syscall) para o Sistema Operacional?

- a) Apenas II e IV.
 - b) Apenas I, II, e V.
 - c) Todas as operações exigem chamadas de sistema, pois o aplicativo inteiro depende do SO para rodar.
 - d) Apenas I, III e IV.
 - e) Apenas II, IV e V.
15. Relacione as afirmações aos respectivos tipos de sistemas operacionais: distribuído (D), multi-usuário (M), desktop (K), servidor (S), embarcado (E) ou de tempo-real (T):
- (a) () Deve ter um comportamento temporal previsível, com prazos de resposta claramente definidos.

- (b) ☐ Sistema operacional usado por uma empresa para executar seu banco de dados corporativo.
- (c) ☐ São tipicamente usados em telefones celulares e sistemas eletrônicos dedicados.
- (d) ☐ Neste tipo de sistema, a localização física dos recursos do sistema computacional é transparente para os usuários.
- (e) ☐ Todos os recursos do sistema têm proprietários e existem regras controlando o acesso aos mesmos pelos usuários.
- (f) ☐ A gerência de energia é muito importante neste tipo de sistema.
- (g) ☐ Sistema que prioriza a gerência da interface gráfica e a interação com o usuário.
- (h) ☐ Construído para gerenciar de forma eficiente grandes volumes de recursos.
- (i) ☐ O MacOS X é um exemplo típico deste tipo de sistema.
- (j) ☐ São sistemas operacionais compactos, construídos para executar aplicações específicas sobre plataformas com poucos recursos.

16. Sobre as afirmações a seguir, relativas aos diversos tipos de sistemas operacionais, indique quais são incorretas, justificando sua resposta:

- (a) Em um sistema operacional de tempo real, a rapidez de resposta é menos importante que a previsibilidade do tempo de resposta.
- (b) Um sistema operacional multi-usuários associa um proprietário a cada recurso do sistema e gerencia as permissões de acesso a esses recursos.
- (c) Nos sistemas operacionais de rede a localização dos recursos é transparente para os usuários.
- (d) Um sistema operacional de tempo real deve priorizar as tarefas que interagem com o usuário.
- (e) Um sistema operacional embarcado é projetado para operar em hardware com poucos recursos.

17. O que são *drivers* de periféricos?

18. Quais das instruções a seguir devem ser deixadas somente em modo núcleo?

- (a) Desabilitar todas as interrupções.
- (b) Ler o relógio da hora do dia.
- (c) Configurar o relógio da hora do dia.
- (d) Mudar o mapa de memória.
- (e) Ler uma porta de entrada/saída.
- (f) Efetuar uma divisão inteira.

- (g) Escrever um valor em uma posição de memória.
 - (h) Ler o valor dos registradores do processador.
19. Considerando um processo em um sistema operacional com proteção de memória entre o núcleo e as aplicações, indique quais das seguintes ações do processo teriam de ser realizadas através de chamadas de sistema, justificando suas respostas:
- (a) Ler o relógio de tempo real do hardware.
 - (b) Enviar um pacote através da rede.
 - (c) Calcular uma exponenciação.
 - (d) Preencher uma área de memória do processo com zeros.
 - (e) Remover um arquivo do disco.
20. Considere um sistema que tem duas CPUs, cada uma tendo duas threads (*hyperthreading*). Suponha que três programas, P0, P1 e P2, sejam iniciados com tempos de execução de 5, 10 e 20 ms, respectivamente. Quanto tempo levará para completar a execução desses programas? Presuma que todos os três programas sejam 100% ligados à CPU, não bloqueiem durante a execução e não mudem de CPUs uma vez escolhidos.
21. Quando um programa de usuário faz uma chamada de sistema para ler ou escrever um arquivo de disco, ele fornece uma indicação de qual arquivo ele quer, um ponteiro para o buffer de dados e o contador. O controle é então transferido para o sistema operacional, que chama o driver apropriado. Suponha que o driver começa o disco e termina quando ocorre uma interrupção. No caso da leitura do disco, obviamente quem chamou terá de ser bloqueado (pois não há dados para ele). E quanto a escrever para o disco? Quem chamou precisa ser bloqueado esperando o término da transferência de disco?
22. O que é uma instrução? Explique o uso em sistemas operacionais.
23. Quais as diferenças entre interrupções, exceções e *traps*?
24. O comando em linguagem C **fopen** é uma chamada de sistema ou uma função de biblioteca? Por quê?
25. Qual tipo de multiplexação (tempo, espaço ou ambos) pode ser usado para compartilhar os seguintes recursos: CPU, memória, disco, placa de rede, impressora, teclado e monitor?
26. As chamadas de sistema abaixo podem retornar qualquer valor em count fora nbytes? Se a resposta for sim, por quê?

```
1      count = read(fd, buffer, nbytes);  
2      count = write(fd, buffer, nbytes);
```

27. Sistemas operacionais modernos desacoplam o espaço de endereçamento do processo da memória física da máquina. Liste duas vantagens desse projeto. (Dica: Espaço de endereçamento são os endereços de memória utilizados pelo processo)
28. Para um programador, uma chamada de sistema parece com qualquer outra chamada para uma rotina de biblioteca. É importante que um programador saiba quais rotinas de biblioteca resultam em chamadas de sistema? Em quais circunstâncias e por quê?
29. Coloque na ordem correta as ações abaixo, que ocorrem durante a execução da função `printf("Hello world")` por um processo (observe que nem todas as ações indicadas fazem parte da sequência).
- (a) A rotina de tratamento da interrupção de software é ativada dentro do núcleo.
 - (b) A função `printf` finaliza sua execução e devolve o controle ao código do processo.
 - (c) A função de biblioteca `printf` recebe e processa os parâmetros de entrada (a string "Hello world").
 - (d) A função de biblioteca `printf` prepara os registradores para solicitar a chamada de sistema `write()`
 - (e) O disco rígido gera uma interrupção indicando a conclusão da operação.
 - (f) O escalonador escolhe o processo mais prioritário para execução.
 - (g) Uma interrupção de software é acionada.
 - (h) O processo chama a função `printf` da biblioteca C.
 - (i) A operação de escrita no terminal é efetuada ou agendada pela rotina de tratamento da interrupção.
 - (j) O controle volta para a função `printf` em modo usuário.
30. Um sistema operacional portátil é um sistema que pode ser levado de uma arquitetura de sistema para outra sem nenhuma modificação. Explique por que é impraticável construir um sistema operacional que seja completamente portátil. Descreva duas camadas de alto nível que você terá ao projetar um sistema operacional que seja altamente portátil.
31. Defina virtualização e diferencie os dois tipos de virtualização vistos. Sugira casos em que cada um possa ser usado.
32. Diferencie a emulação da virtualização e da tradução de programas.
33. Monte uma tabela com os benefícios e deficiências mais relevantes das principais arquiteturas de sistemas operacionais.
34. Sobre as afirmações a seguir, relativas às diversas arquiteturas de sistemas operacionais, indique quais são incorretas, justificando sua resposta:

- (a) Uma máquina virtual de sistema é construída para suportar uma aplicação escrita em uma linguagem de programação específica, como Java.
 - (b) Um hipervisor convidado executa sobre um sistema operacional hospedeiro.
 - (c) Em um sistema operacional micronúcleo, os diversos componentes do sistema são construídos como módulos interconectados executando dentro do núcleo.
 - (d) Núcleos monolíticos são muito utilizados devido à sua robustez e facilidade de manutenção.
 - (e) Em um sistema operacional micronúcleo, as chamadas de sistema são implementadas através de trocas de mensagens.
35. Quais são os três estados básicos de um processo? Na teoria, com três estados, poderia haver seis transições, duas para cada. No entanto, apenas quatro transições são usadas. Quais são elas? Existe alguma circunstância na qual uma delas ou ambas as transições perdidas possam ocorrer? Por que? Desenhe o diagrama de estados.
36. Relacione as afirmações abaixo aos respectivos estados no ciclo de vida das tarefas (N: Nova, P: Pronta, E: Executando, S: Suspensa, T: Terminada):
- (a) O código da tarefa está sendo carregado.
 - (b) As tarefas são ordenadas por prioridades.
 - (c) A tarefa sai deste estado ao solicitar uma operação de entrada/saída.
 - (d) Os recursos usados pela tarefa são devolvidos ao sistema.
 - (e) A tarefa vai a este estado ao terminar seu quantum.
 - (f) A tarefa só precisa do processador para poder executar.
 - (g) O acesso a um semáforo em uso pode levar a tarefa a este estado.
 - (h) A tarefa pode criar novas tarefas.
 - (i) Há uma tarefa neste estado para cada processador do sistema.
 - (j) A tarefa aguarda a ocorrência de um evento externo.
37. Presuma que você esteja tentando baixar um arquivo grande de 2 GB da internet. O arquivo está disponível a partir de um conjunto de servidores espelho, cada um deles capaz de fornecer um subconjunto dos bytes do arquivo; presuma que uma determinada solicitação especifique os bytes de início e fim do arquivo. Explique como você poderia usar os threads para melhorar o tempo de download.
38. Diferencie tabela de processos de TCB e responda: qual a necessidade de possuir ambas em um sistema operacional?
39. Com base nos conceitos relacionados ao sistema operacional, analise as afirmativas a seguir.

- I. O sistema operacional é responsável por permitir a interação entre o usuário, os programas e os componentes físicos do computador.
- II. Durante a inicialização do computador, o sistema operacional é carregado para a memória a fim de controlar os recursos disponíveis.
- III. A lentidão percebida após ligar o computador pode estar relacionada à quantidade de programas que o sistema operacional carrega automaticamente.
- IV. Um sistema operacional atua apenas no momento em que o computador é ligado, deixando de interferir após a abertura dos programas.
- V. A execução simultânea de dois programas depende exclusivamente da existência de mais de um núcleo no processador.

Assinale a alternativa correta:

- (a) Apenas as afirmativas I, II, III e V são verdadeiras.
- (b) Apenas as afirmativas I, II e III são verdadeiras.
- (c) Apenas as afirmativas II, III, IV e V são verdadeiras.
- (d) Apenas as afirmativas I, III e V são verdadeiras.

40. Desenhe o diagrama de tempo da execução do código a seguir e calcule a duração aproximada de sua execução.

```

1      int main() {
2          if( fork () == 0 ) {
3              if( fork () == 0 ) {
4                  sleep(2);
5                  exit(0);
6              }
7              else {
8                  sleep(3);
9                  exit(0);
10             }
11         }
12         else {
13             if( fork () == 0 ) {
14                 sleep(4);
15                 exit(0);
16             }
17             else {
18                 sleep(2);
19                 exit(0);
20             }
21         }

```

```

22
23         if( fork ( ) == 0 ) {
24             sleep(2);
25             exit(0);
26         }
27         else {
28             sleep(6);
29             exit(0);
30         }
31     }

```

41. O que são threads e para que servem?
42. Escreva um código em C que receba uma quantidade N e um valor M. Crie então N *threads*, de forma que a primeira conte de 0 a M/N, a segunda conte de M/N + 1 a 2 × M/N etc... Ao final imprima o valor final obtido (soma de todos os resultados das threads) e o valor esperado.
43. Quais as principais vantagens e desvantagens de threads em relação a processos?
44. Forneça exemplos de problemas cuja implementação multi-thread não tem desempenho melhor que a respectiva implementação sequencial.
45. Como threads podem ser usadas para melhorar o desempenho de um servidor de arquivos?
46. Neste problema, você deve comparar a leitura de um arquivo usando um servidor de arquivos de um thread único e um servidor com múltiplos threads. São necessários 12 ms para obter uma requisição de trabalho, despachá-la e realizar o resto do processamento, presumindo que os dados necessários estejam na cache de blocos. Se uma operação de disco for necessária, como é o caso em um terço das vezes, 75 ms adicionais são necessários, tempo em que o thread repousa. Quantas requisições/segundo o servidor consegue lidar se ele tiver um único thread? E se ele for multithread?
47. Existe alguma maneira de se forçar que a ordem de um programa seja estritamente: thread 1 criado, thread 1 imprime mensagem, thread 1 sai, thread 2 criado, thread 2 imprime mensagem, thread 2 sai e assim por diante? Se a resposta for afirmativa, como? Se não, por que não?
48. Considere o fragmento de código C seguinte:

```

1     void main( ) {
2         fork( );
3         fork( );
4         fork( );

```

```

5         exit( );
6     }

```

Quantos processos filhos são criados com a execução desse programa?

49. É possível determinar se um processo é propenso a se tornar limitado pela CPU (*CPU-bound*) ou limitado pela E/S (*I/O bound*) analisando o código fonte? Como isso pode ser determinado no tempo de execução?
50. Explique o que é um quantum. Como e com base em que critérios é escolhida a duração de um quantum de processamento?
51. O que é feito durante um chaveamento de contexto? O tempo empregado é irrelevante?
52. Explique o que é escalonamento round-robin, dando um exemplo.
53. No problema do jantar dos filósofos, deixe o protocolo a seguir ser usado: um filósofo de número par sempre pega o seu garfo esquerdo antes de pegar o direito; um filósofo de número ímpar sempre pega o garfo direito antes de pegar o esquerdo. Esse protocolo vai garantir uma operação sem impasse?
54. Qual a diferença do problema dos produtores e consumidores para o problema dos leitores e escritores? Cite exemplos reais que são modelados por esses problemas.
55. A tabela a seguir representa um conjunto de tarefas prontas para utilizar um processador:

Tarefa	t_1	t_2	t_3	t_4	t_5
ingresso	0	0	3	5	7
duração	5	4	5	6	4
prioridade	2	3	5	9	6

Represente graficamente a sequência de execução das tarefas e calcule os tempos médios de vida (*turnaround time*) e de espera (*waiting time*), para as políticas de escalonamento a seguir:

- (a) FCFS
- (b) RR
- (c) RR com Prioridades

Assuma um quantum de 1 ms para o RR.

56. Faça o mesmo para as tarefas abaixo:

Tarefa	t_1	t_2	t_3	t_4	t_5
ingresso	0	0	1	7	11
duração	5	6	2	6	4
prioridade	2	3	4	7	9

57. Repita o processo da questão anterior para um sistema com dois núcleos. Dica: Mantenha filas de prontos para cada prioridade, onde o processo em execução no momento está ao final da fila e o próximo a executar está no início.

58. Para um escalonamento RR puro com quantum Q, e uma quantidade N de processos, quanto tempo levará para que um processo que acabou de executar possa executar novamente? Se o processo precisa de 5 quantums para finalizar, qual o tempo total entre sua criação e sua finalização? Suponha que a primeira execução aconteça no instante da sua criação.

59. Suponha que um sistema operacional em que 7 processos sejam executados conforme mostra a tabela abaixo. Desenhe o diagrama de tempo até o instante 30 ms, levando em consideração um quantum de 1 ms. Utilize um escalonamento Round Robin (RR) com prioridades e preemptivo.

Tarefa	Instante de Criação	Prioridade	Instante de Bloqueio	Instante de Retorno
A	0 ms	2	2 ms	7 ms
B	0 ms	2	4 ms	7 ms
C	0 ms	1	-	-
D	6 ms	3	-	-
E	9 ms	3	14 ms	16 ms
F	13 ms	2	16 ms	20 ms
G	19 ms	4	22 ms	26 ms

60. Refaça para um quantum de 2 ms.

61. Refaça utilizando dois núcleos ao invés de um, usando um quantum de 1 ms.

62. Explique o que são condições de corrida e regiões críticas, mostrando um exemplo real.

63. Sobre as afirmações a seguir, relativas aos mecanismos de coordenação, indique quais são incorretas, justificando sua resposta:

- (a) A estratégia de inibir interrupções para evitar condições de disputa funciona em sistemas multi-processados.
 - (b) Os mecanismos de controle de entrada nas regiões críticas provêem exclusão mútua no acesso às mesmas.
 - (c) Os algoritmos de busy-wait se baseiam no teste contínuo de uma condição.
 - (d) Condições de disputa ocorrem devido às diferenças de velocidade na execução dos processos.
 - (e) Condições de disputa ocorrem quando dois processos tentam executar o mesmo código ao mesmo tempo.
 - (f) Instruções do tipo TSL (Test & Set Lock) devem ser implementadas pelo núcleo do SO.
 - (g) O algoritmo de Peterson garante justiça no acesso à região crítica.
 - (h) Os algoritmos com estratégia espera ocupada otimizam o uso da CPU do sistema.
 - (i) Uma forma eficiente de resolver os problemas de condição de disputa é introduzir pequenos atrasos nos processos envolvidos.
64. Explique o que é espera ocupada e por que os mecanismos que empregam essa técnica são considerados ineficientes.
65. Por que não existem operações **read(s)** e **write(s)** para ler ou ajustar o valor atual de um semáforo?
66. O que são operações atômicas?
67. Mostre como pode ocorrer violação da condição de exclusão mútua se as operações **down(s)** e **up(s)** sobre semáforos não forem implementadas de forma atômica.
68. Em que situações um semáforo deve ser inicializado em 0, 1 ou $n > 1$?
69. Qual a diferença entre semáforos binários e contadores? E mutex?
70. Como funcionam as barreiras?
71. Sobre as arquiteturas de Sistemas Operacionais, qual a principal característica de um sistema monolítico?
- a) O sistema operacional é dividido em pequenos módulos que executam no espaço de usuário, comunicando-se por troca de mensagens.
 - b) Todo o núcleo do sistema roda em modo privilegiado como um único grande programa, sem restrições de acesso entre seus componentes.
 - c) Utiliza o conceito de hipervisor para virtualizar o hardware antes de carregar o sistema principal.

- d) Delega a maior parte de suas funções para um exonúcleo que compartilha recursos dinamicamente com as aplicações.
- e) Baseia-se exclusivamente em contêineres para isolar processos críticos do sistema operacional.

72. Nos sistemas baseados em Micronúcleo (Microkernel), uma das principais vantagens é:

- a) O alto desempenho devido à ausência de troca de mensagens entre processos.
- b) A ausência total de chamadas de sistema (syscalls).
- c) Maior confiabilidade e segurança, pois a maioria dos serviços roda no espaço de usuário, isolados uns dos outros.
- d) A execução de todo o código do sistema operacional no modo supervisor (kernel mode).
- e) A impossibilidade de ocorrerem travamentos do sistema (kernel panics).

73. Em relação à virtualização, qual a principal diferença entre Máquinas Virtuais (VMs) e Contêineres?

- a) VMs compartilham o núcleo do SO hospedeiro, enquanto contêineres exigem um hipervisor tipo 1.
- b) Contêineres emulam o hardware físico, enquanto VMs apenas emulam o espaço de usuário.
- c) VMs incluem seu próprio sistema operacional convidado (Guest OS), enquanto contêineres compartilham o mesmo núcleo (Kernel) do SO hospedeiro, isolando o espaço de usuário.
- d) Não há diferença, ambos são implementações do conceito de sistemas monolíticos.
- e) Contêineres são mais lentos e consomem mais memória do que Máquinas Virtuais.

74. O que define um Hipervisor do tipo Nativo (*Bare-metal*)?

- a) Executa sobre um sistema operacional hospedeiro convencional (ex: Windows ou Linux).
- b) É implementado inteiramente em hardware, sem necessidade de software.
- c) Executa diretamente sobre o hardware do hospedeiro, substituindo a necessidade de um sistema operacional tradicional na camada base.
- d) Só suporta a execução de aplicações isoladas em Java ou C#.
- e) É restrito apenas à virtualização de redes e dispositivos de armazenamento.

75. Qual a principal diferença conceitual entre um Programa e um Processo?

- a) O programa é dinâmico e tem estado interno, enquanto o processo é estático.

- b) Um programa é uma entidade passiva (código no disco), enquanto o processo é a entidade ativa, com estado interno, em execução.
- c) Não há diferença prática na arquitetura de sistemas operacionais modernos.
- d) Processos são utilizados apenas em sistemas de tempo real, programas em sistemas de lote.
- e) Programas residem na memória cache, enquanto processos ficam exclusivamente no disco rígido.

76. O Bloco de Controle de Tarefa (TCB) é uma estrutura fundamental para o Sistema Operacional. Qual das informações abaixo NÃO costuma ser mantida no TCB?

- a) O estado atual do processo.
- b) Os valores dos registradores da CPU (contexto de hardware).
- c) O código-fonte original em linguagem de alto nível do aplicativo.
- d) O valor do contador de programa (Program Counter - PC).
- e) Informações sobre arquivos abertos pelo processo.

77. O que ocorre quando um processo muda do estado “Executando” para o estado “Bloqueado”?

- a) O processo finalizou a sua execução e será retirado da memória.
- b) O processo sofreu preempção pelo escalonador devido ao término de seu quantum de tempo.
- c) O processo precisou aguardar por um evento externo, como a conclusão de uma operação de Entrada/Saída.
- d) O processo detectou uma condição de corrida e paralisou os demais processos.
- e) O processo requisitou a criação de uma nova thread de núcleo.

78. Por que a criação e a troca de contexto entre Threads do mesmo processo é geralmente mais rápida do que entre Processos distintos?

- a) Porque threads executam diretamente no hardware sem passar pelo Sistema Operacional.
- b) Porque threads não possuem contexto de registradores para salvar.
- c) Porque as threads de um mesmo processo compartilham o mesmo espaço de endereçamento (memória), evitando o recarregamento pesado de tabelas de páginas.
- d) Porque threads são gerenciadas obrigatoriamente por hardware.
- e) Porque a CPU aloca um núcleo exclusivo para cada thread instanciada.

79. Em uma aplicação multithread, quais componentes do contexto de execução são estritamente individuais (privados) para cada thread e não compartilhados com as demais threads do mesmo processo?
- a) Variáveis globais e arquivos abertos.
 - b) O código executável e a seção de dados.
 - c) A memória heap e as conexões de rede.
 - d) O Contador de Programa (PC), os Registradores e a Pilha (Stack).
 - e) Os descritores de segurança e variáveis estáticas.
80. Na gestão de processos, o que significa o conceito de “Preempção” em um algoritmo de escalonamento?
- a) A capacidade do usuário cancelar a execução de um processo.
 - b) A propriedade do SO de interromper um processo em execução, retirar a CPU dele e atribuí-la a outro processo.
 - c) A garantia de que processos longos sempre terminarão antes de processos curtos.
 - d) A alocação estática de memória baseada no tempo de execução previsto.
 - e) A execução cooperativa onde o processo escolhe quando liberar a CPU.
81. Nos algoritmos de escalonamento focados em sistemas iterativos (iterativos), uma métrica essencial é o Tempo de Resposta. Qual algoritmo abaixo é caracterizado por atribuir uma fatia de tempo (quantum) para cada processo em uma fila circular?
- a) First-Come First-Served (FCFS).
 - b) Shortest Job First (SJF).
 - c) Escalonamento garantido.
 - d) Round-Robin.
 - e) Nenhum dos algoritmos acima.
82. Quando duas ou mais tarefas acessam recursos compartilhados ao mesmo tempo e o resultado final depende da ordem temporal exata da execução das instruções, temos a ocorrência de:
- a) Uma Condição de Disputa (Race Condition).
 - b) Um Chaveamento de Contexto.
 - c) Uma Multiplexação no Espaço.
 - d) Um Hipervisor.
 - e) Um Escalonamento Iterativo.

83. Para garantir a exclusão mútua na Seção Crítica sem intervenção do SO (abordagens de software e hardware), o uso de Espera Ocupada (Busy Waiting) possui a desvantagem de:
- a) Impedir completamente a ocorrência de preempções.
 - b) Causar falhas de segmentação de memória nas threads inativas.
 - c) Desperdiçar ciclos úteis da CPU enquanto a tarefa apenas testa repetidamente se pode entrar na região crítica.
 - d) Forçar a reinicialização automática do módulo de escalonamento.
 - e) Ser inútil em processadores de um único núcleo.
84. Qual a utilidade de instruções de máquina específicas como o TSL (Test and Set Lock) ou CAS (Compare and Swap)?
- a) Permitem apagar os registradores atômicos durante exceções do SO.
 - b) Permitem ler e modificar o valor de uma variável de memória de forma atômica, servindo de base mecânica para construir travas de exclusão mútua.
 - c) São usadas para forçar o fechamento de contêineres e máquinas virtuais.
 - d) Desabilitam irreversivelmente as interrupções de hardware da placa mãe.
 - e) São usadas exclusivamente para escalonar threads de usuário em sistemas antigos.
85. A inibição (desabilitação) de interrupções é uma forma rudimentar de garantir exclusão mútua. No entanto, sua principal falha arquitetural em sistemas modernos é que:
- a) Ela permite que o usuário crie ilimitadas chamadas de sistema.
 - b) O sistema consumirá todo o disco rígido com logs de interrupção.
 - c) Não funciona adequadamente em sistemas multiprocessadores, pois inibe interrupções em apenas um núcleo, não evitando o acesso paralelo ao recurso por outro processador.
 - d) Elimina a necessidade de escalonadores em sistemas interativos.
 - e) Aumenta a memória virtual exponencialmente enquanto está ativada.
86. Um Semáforo, proposto por Dijkstra, é um mecanismo de coordenação baseado em:
- a) Variáveis booleanas simples que requerem espera ocupada em loops (while).
 - b) Um contador inteiro protegido e uma fila de processos associada, acessado pelas operações atômicas Down (P) e Up (V).
 - c) Blocos de memória RAM alocados fisicamente para comunicação em rede.
 - d) Instruções de hardware exclusivas de arquiteturas RISC que disparam interrupções de disco.

- e) Estruturas de linguagem orientada a objetos que incluem métodos implícitos para os recursos.

87. Na operação clássica **Down(S)** (ou **P**) em um semáforo:

- a) O contador do semáforo é incrementado, e se for ≤ 0 , agenda uma tarefa da fila.
- b) O valor de **S** é zerado independentemente de seu valor anterior.
- c) O contador do semáforo é decrementado. Se o valor for menor que zero, a tarefa requisitante é bloqueada e inserida na fila do semáforo.
- d) A exclusão mútua é permanentemente ignorada para a tarefa que efetuou a chamada.
- e) O processo é destruído caso tente acessar a região crítica fora do tempo previsto.

88. O **Mutex** (Mutual Exclusion lock) pode ser visto como um caso especial de Semáforo. Qual a sua característica de implementação?

- a) É um semáforo binário focado puramente em exclusão mútua de um recurso único, assumindo estados equivalentes a “bloqueado” ou “livre”.
- b) É um semáforo que permite valores negativos infinitos.
- c) É uma ferramenta para gerar múltiplas instâncias de tarefas simultâneas.
- d) Diferente do semáforo, o **Mutex** só pode ser usado por threads e nunca por processos de núcleo.
- e) **Mutexes** obrigatoriamente exigem o uso de espera ocupada intensiva.

89. Em relação aos Monitores, assinale a afirmação correta:

- a) São peças de hardware de exibição visual acopladas diretamente na placa-mãe.
- b) São construtos de linguagens de programação (estruturas de alto nível) que garantem que apenas uma tarefa possa executar dentro dos seus métodos a cada instante, encapsulando os dados e a exclusão mútua.
- c) São algoritmos teóricos impossíveis de serem implementados em Java ou C.
- d) Funcionam estritamente por meio da desabilitação manual de interrupções por parte do programador.
- e) Substituem a necessidade do Sistema Operacional possuir um mecanismo de escalonamento de CPU.

90. No contexto de um Monitor, as “Variáveis de Condição” (**condition variables**) operadas via **wait** e **signal** servem para:

- a) Ajustar o nível de brilho gráfico do monitor de vídeo do terminal.

- b) Bloquear um processo dentro do monitor e liberar temporariamente a exclusão mútua para que outro processo possa entrar e eventualmente alterar a condição esperada.
- c) Interromper forçosamente processos de baixo privilégio em favor de processos root.
- d) Criar novas threads filhas de forma assíncrona.
- e) Aumentar iterativamente o contador de acessos do TCB.

91. No problema clássico do “Produtor/Consumidor”, o uso de semáforos é essencial para:

- a) Garantir exclusão mútua no acesso ao buffer, e para sincronizar o produtor (caso o buffer esteja cheio) e o consumidor (caso o buffer esteja vazio).
- b) Criar uma barreira temporal que força produtores a trabalharem mais devagar que consumidores em tempo de execução.
- c) Transformar o buffer finito em um recurso alocado direto do disco rígido local.
- d) Evitar que as threads transitem do modo kernel de volta ao modo de usuário acidentalmente.
- e) Excluir consumidores extras se eles ultrapassarem a prioridade de execução.

92. Sobre o problema do “Jantar dos Filósofos”, qual a principal condição problemática que o algoritmo de solução deve prevenir rigorosamente?

- a) Preempção excessiva pelo Round-Robin gerando fome por CPU (Starvation).
- b) A sobrecarga de chamadas de rede no escalonador estático do Kernel.
- c) A ocorrência de Impasse (Deadlock) quando todos os filósofos decidem pegar apenas um palito (ex: o da direita) simultaneamente, não havendo mais palitos livres.
- d) Filósofos falarem uns com os outros gerando overhead de IPC (Inter-process Communication).
- e) Consumidores roubando a porção do produtor no buffer da mesa circular.

93. Qual a principal diretriz no problema clássico de coordenação dos “Leitores/Escretores”?

- a) Múltiplos escritores podem modificar o arquivo simultaneamente para ganhar desempenho.
- b) Permitir leituras concorrentes por múltiplos leitores, mas garantir que acessos de escrita sejam estritamente mutuamente exclusivos com relação a qualquer outra leitura ou escrita.
- c) Permitir que a escrita só aconteça após o processo de leitura falhar completamente por tempo esgotado.

- d) Forçar que todas as interrupções de teclado sejam redirecionadas para o núcleo leitor mais veloz.
 - e) Utilizar um semáforo único que proíbe totalmente a concorrência entre leitores.
94. Numa solução básica para o Jantar dos Filósofos que utiliza semáforos independentes para cada palito, se a ordem de requisição dos palitos por parte de TODOS os filósofos for idêntica (ex: sempre pega o esquerdo primeiro, e o direito depois), o sistema está sujeito ao risco imediato de:
- a) Erro de divisão por zero na ALU.
 - b) Impasse (*Deadlock*).
 - c) Aumento de throughput.
 - d) Falha de cache L1.
 - e) Desabilitação completa das interrupções.
95. Considere a implementação de coordenação com “Exclusão Mútua Através de Troca de Variáveis” (Alternância Estrita). A principal desvantagem dessa abordagem lógica (baseada no teste `turn == other`) é:
- a) Causa preempção obrigatória do hardware antes da avaliação terminar.
 - b) Utiliza operações atômicas custosas da placa de rede.
 - c) Força os processos a entrarem na região crítica em ordem rigorosamente alternada; um processo veloz pode ser injustamente forçado a esperar um processo lento que sequer precisa entrar na região crítica.
 - d) É uma abordagem estritamente limitada a sistemas distribuídos globais através da internet.
 - e) Garante a solução para problemas em nível de hardware, mas falha inevitavelmente no escalonamento interativo SJF.
96. Nos sistemas operativos interativos (como o Windows ou o Linux que usamos no dia a dia num computador pessoal), qual é o objetivo principal que o escalonador de processos tenta atingir?
- a) Maximizar o uso do disco rígido gravando o máximo de ficheiros possíveis simultaneamente.
 - b) Executar os processos mais longos primeiro, deixando as tarefas rápidas para o final.
 - c) Minimizar o tempo de resposta, para que o utilizador sinta que o sistema é rápido e fluido ao clicar ou digitar.
 - d) Garantir que cada processo criado receba exatamente uma hora contínua de uso do processador.

- e) Evitar completamente o uso da memória RAM, correndo os processos diretamente do disco.

97. O algoritmo Round-Robin (Escalonamento Circular) é um dos mais antigos e famosos para sistemas interativos. Como funciona de forma básica?

- a) O processo ganha o processador e executa até terminar todo o seu código, sem poder ser interrompido.
- b) O sistema operativo escolhe o próximo processo de forma totalmente aleatória, atirando um "dado" interno.
- c) O processo com o menor tamanho no disco passa sempre à frente de todos os outros.
- d) O sistema operativo pergunta manualmente ao utilizador que programa deseja correr a cada 5 segundos.
- e) Cada processo ganha um pequeno intervalo de tempo (chamado de *quantum*). Se não terminar nesse tempo, é interrompido e volta para o final da fila.

98. Ainda sobre o algoritmo Round-Robin, o que acontece se o Sistema Operativo configurar um *quantum* (fatia de tempo) muito pequeno, como por exemplo 1 milissegundo?

- a) O sistema sofrerá com um excesso de trocas de contexto, desperdiçando muito tempo da CPU apenas para trocar de processos constantemente.
- b) O computador correrá muito mais rápido, gastando menos energia elétrica.
- c) O sistema transformar-se-á automaticamente num sistema de processamento em lote (batch).
- d) A memória RAM do computador terá o seu espaço dobrado magicamente.
- e) Os processos correrão tão rápido que o ecrã do computador não conseguirá acompanhá-los.

99. No "Escalonamento por Prioridades", os processos recebem níveis de importância diferentes. Sobre este mecanismo em sistemas interativos, qual a afirmação correta?

- a) Todos os processos recebem sempre a prioridade máxima para que o sistema seja perfeitamente igualitário.
- b) Processos que ocupam mais espaço no disco rígido recebem, por predefinição, a maior prioridade.
- c) Apenas processos do Sistema Operativo podem ter prioridade; os programas do utilizador não entram no escalonador.

- d) O processo de maior prioridade ganha a CPU. Porém, o SO geralmente diminui essa prioridade aos poucos enquanto este corre, para evitar que processos de baixa prioridade nunca executem (fome / *starvation*).
 - e) É um algoritmo usado exclusivamente para gerir a fila de impressão, e não o processador.
100. Uma variação interessante para escalonamento interativo é o algoritmo de “Escalonamento por Loteria” (*Lottery Scheduling*). Qual é a lógica principal deste método?
- a) Os programas devem pagar uma taxa em criptomoedas ao SO para conseguirem correr.
 - b) O SO distribui “bilhetes” virtuais aos processos. Quando chega a hora de escolher quem vai usar a CPU, um bilhete é sorteado e o processo dono do mesmo ganha a vez.
 - c) O utilizador precisa de acertar num mini-jogo no ecrã para autorizar a execução de uma aplicação.
 - d) É um sistema onde é totalmente impossível dar vantagem a um processo mais importante que os outros.
 - e) Funciona distribuindo a CPU apenas para os processos que acabaram de ser descarregados da internet.

Bons estudos!